

Semana 2 — CharacterBody3D, movimentação, Input Map e Câmera

Semana 2

CharacterBody3D, movimentação, Input Map e Câmera

Disciplina: Tendências de Motores de Jogos (IN46A)

Curso: Tecnologia em Jogos Digitais — IFMS Campus Dourados

Unidade I — Aprender a Ferramenta (Semanas 1–3)

Projeto: Vertical Slice *O Templo Esquecido*

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Objetivos da Semana

- Compreender por que uma engine desacopla a intenção do jogador da ação no mundo
- Configurar um CharacterBody3D controlável usando `move_and_slide`
- Configurar um Input Map e conectar Actions à movimentação do Player
- Corrigir o `Chao` para ter colisão física real, distinguindo `MeshInstance3D` (visual) de `StaticBody3D` (física)
- Configurar uma câmera em terceira pessoa (`SpringArm3D` + `Camera3D`) que acompanha o Player sem atravessar paredes
- Ajustar o movimento para ser relativo à direção da câmera, não a eixos fixos do mundo

Semana 2 — CharacterBody3D, movimentação, Input Map e Câmera

ENCONTRO 1

CharacterBody3D e move_and_slide

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 1

- Revisão da Scene da Semana 1 (10 min)
- Introdução: por que engines desacoplam intenção de ação (15 min)
- Demonstração: CharacterBody3D + CollisionShape3D via Orchestrator (35 min)
- Laboratório: cada estudante monta seu Player (45 min)
- Desafio: ajustar forma de colisão (20 min)
- Feedback e fechamento (10 min)

Semana 2 — CharacterBody3D, movimentação, Input Map e Câmera



Como um personagem sabe que não deve atravessar uma parede?

Pense em qualquer jogo que vocês já jogaram, de qualquer engine.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

O Problema Universal da Locomoção

Todo personagem controlável — jogador ou inimigo — precisa resolver o mesmo problema físico:

- Mover-se no mundo sem atravessar paredes
- Deslizar suavemente ao encostar em obstáculos
- Responder corretamente a rampas e desníveis

Reimplementar isso do zero a cada projeto seria caro e repetitivo.

CharacterBody3D e move_and_slide

- **CharacterBody3D** — Node especializado em corpos controlados por código
- Diferente do RigidBody3D, que é simulado livremente pela física
- `move_and_slide()` — resolve deslocamento e colisão em uma única chamada, a cada frame de física
- Ao final deste encontro, o Player existe e é sólido — mas ainda não se move sozinho

Locomoção no Godot × Unity

Godot

- CharacterBody3D já pronto
- `move_and_slide()` resolve tudo em uma chamada
- Solução de locomoção embutida no próprio Node

Unity

- CharacterController **ou** Rigidbody + script próprio
- Sem um único componente "pronto" equivalente
- Mais decisões de arquitetura recaem sobre o time

Demonstração — Montagem do Player

O que será construído:

- Node `Player` (`CharacterBody3D`), filho de `NívelTeste`
- `CollisionShape3D` — forma física de colisão
- `Malha` (`MeshInstance3D`) — representação visual
- Orchestration `player.torch` chamando `move_and_slide` no `PhysicsProcess`
- `Player` salvo como Scene própria (`scenes/characters/Player.tscn`), reaproveitável em outros níveis

Por quê: primeiro personagem controlável do semestre, base direta do Input Map no Encontro 2.

Referência em GDScript — Encontro 1

```
extends CharacterBody3D

func _physics_process(delta: float) -> void:
    move_and_slide()
```

Isso é tudo que a Orchestration `player.torch` precisa fazer neste encontro: um único nó de evento `PhysicsProcess` chamando `move_and_slide`, sem nenhuma leitura de input ainda.

Imagem sugerida

*Objetivo: mostrar a árvore de cena com **Player** (CharacterBody3D) já contendo **CollisionShape3D** e **Malha**, ao lado do Viewport 3D com a cápsula posicionada sobre o **Chão**.*

Enquadramento: captura de tela dividida — Scene dock à esquerda, Viewport 3D à direita.

*Elementos presentes: hierarquia **Player** > **CollisionShape3D**, **Malha**, cápsula alinhada ao chão no Viewport.*

Destaque visual: contorno colorido separando forma de colisão (vermelho, semi-transparente) da malha visual (azul).

Legenda sugerida: "Player montado: colisão e malha visual como Nodes separados, alinhados sobre o chão."

Laboratório — Montagem do Player

Cada estudante replica, no próprio projeto:

1. `Player` (`CharacterBody3D`) como filho de `NivelTeste`
2. `CollisionShape3D` com uma Shape atribuída (ex.: `CapsuleShape3D`)
3. `Malha` (`MeshInstance3D`) com uma Mesh atribuída, alinhada à colisão
4. Orchestration `player.torch` chamando `move_and_slide` no `PhysicsProcess`
5. `Player` salvo como Scene própria em `scenes/characters/Player.tscn`

Boas Práticas — Colisão e Composição

- Separar sempre `CollisionShape3D` (física) de `MeshInstance3D` (visual), mesmo quando parecem redundantes
- Nomear o Node raiz como `Player`, nunca deixar o nome padrão `CharacterBody3D`
- Confirmar visualmente o alinhamento entre colisão e malha antes de avançar
- Associar a Orchestration ao Node correto (`Player`, não `NívelTeste`)



Desafio — Encontro 1

Ajuste a forma ou o tamanho da `CollisionShape3D` do próprio Player — por exemplo, cápsula versus caixa, ou uma escala diferente da demonstrada.

OBJETIVOS

Justifique brevemente a escolha em relação ao personagem que pretende usar no Vertical Slice. Não há solução única.

Fechamento — Encontro 1

- Player (CharacterBody3D) montado, sólido, alinhado ao `Chao`
- `move_and_slide` já chamado a cada frame de física, ainda sem velocity
- Próximo passo: conectar o Input Map à movimentação, no Encontro 2

Semana 2 — CharacterBody3D, movimentação, Input Map e Câmera

ENCONTRO 2

Input Map, InputEvent e Câmera em Terceira Pessoa

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Agenda do Encontro 2

- Revisão do Encontro 1 (Player parado) (10 min)
- Introdução: por que desacoplar dispositivo físico de ação lógica (15 min)
- Demonstração: Input Map + conexão com move_and_slide (25 min)
- Laboratório: cada estudante configura o próprio Input Map (30 min)
- Introdução + Demonstração: correção do `Chao` (`StaticBody3D`) e câmera em terceira pessoa (`SpringArm3D` + `Camera3D`) (20 min)
- Laboratório: cada estudante corrige o `Chao` e configura a própria câmera (20 min)
- Desafio: Action adicional (correr, agachar ou pular) (10 min)
- Feedback e fechamento (5 min)

Semana 2 — CharacterBody3D, movimentação, Input Map e Câmera



Se o código do jogo checasse diretamente a tecla W, o que aconteceria ao tentar suportar um gamepad?

Pense antes de abrir o Project Settings.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Input Map — Camada de Desacoplamento

Se o código lesse diretamente "tecla W pressionada", qualquer remapeamento exigiria reescrever a lógica do jogo inteira.

- O jogador aperta uma tecla física
- A engine traduz isso em uma **Action** nomeada (ex.: "mover para frente")
- O código de gameplay consome a Action — nunca a tecla em si

Input Map e InputEvent no Godot

- **Input Map** — configurado uma vez em Project Settings, associa teclas a Actions
- **InputEvent** — evento gerado a cada interação do jogador, traduzido automaticamente para a Action
- A Orchestration só pergunta: "a Action `move_forward` está pressionada agora?"
- Actions desta semana: `move_forward`, `move_back`, `move_left`, `move_right`

Input Map no Godot × Unity

Godot

- Input Map único e global do projeto
- Actions configuradas em um só lugar
- Mais simples para o caso de um único jogador

Unity

- Input System com Action Maps
- Componente Player Input conecta Actions ao código
- Mais granularidade para múltiplos esquemas/dispositivos

Diagrama Sugerido — Fluxo do Input

Diagrama sugerido

Fluxo linear: `Tecla física (W)` → `InputEvent` → `Action (move_forward)` → `Orchestration`
`lê a Action` → `velocity atribuída` → `move_and_slide()`.

Objetivo: visualizar as camadas entre o dispositivo físico e o movimento efetivo do Player.

Legenda sugerida: "Da tecla física ao movimento: cada seta é uma camada de desacoplamento."

Demonstração — Input Map e Conexão

O que será construído:

- Quatro Actions no Input Map: `move_forward`, `move_back`, `move_left`, `move_right`
- Leitura das Actions dentro da Orchestration `player.torch`
- Combinação em um vetor de direção, aplicado à `velocity` do Player
- `move_and_slide` chamado após a `velocity` ser atribuída

Por quê: transforma o Player sólido do Encontro 1 em um Player efetivamente controlável.

Referência em GDScript — Encontro 2

```
extends CharacterBody3D

const SPEED := 5.0

func _physics_process(delta: float) -> void:
    var input_dir := Input.get_vector("move_left", "move_right", "move_forward",
    "move_back")

    velocity.x = input_dir.x * SPEED
    velocity.z = input_dir.y * SPEED

    move_and_slide()
```

Este código não deve ser digitado — é a referência para pensar em como organizar os nós no Orchestrator: leitura de cada Action, combinação em um vetor de direção, multiplicação pela velocidade e atribuição à `velocity` antes de `move_and_slide`.

Imagem sugerida

Objetivo: mostrar a aba Input Map do Project Settings, com as quatro Actions de movimentação já configuradas.

Enquadramento: captura de tela da janela Project Settings, aba Input Map.

Elementos presentes: lista de Actions (`move_forward`, `move_back`, `move_left`, `move_right`), teclas associadas a cada uma.

Destaque visual: o botão "Add" usado para criar uma nova Action.

Legenda sugerida: "Input Map do projeto com as Actions de movimentação configuradas."

Laboratório — Input Map e Movimentação

Cada estudante configura, no próprio projeto:

1. Quatro Actions no Input Map (`move_forward` , `move_back` , `move_left` , `move_right`)
2. Leitura das Actions na Orchestration `player.torch`
3. Vetor de direção combinado e aplicado à `velocity`
4. Teste com F6, movendo o Player nas quatro direções

Boas Práticas — Nomenclatura de Input

- Nomear Actions por intenção do jogador (`move_forward`), nunca pela tecla física
- Centralizar toda leitura de input do Player dentro da própria Orchestration
- Testar cada Action isoladamente antes de combinar as quatro em um vetor
- Evitar espalhar chamadas de Input Map por múltiplos Nodes

Semana 2 — CharacterBody3D, movimentação, Input Map e Câmera



Se a câmera simplesmente ficasse presa atrás do personagem, o que aconteceria ao encostar numa parede?

Pense antes de configurarmos a câmera no editor.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Câmera em Terceira Pessoa — Problema Universal

Toda câmera que segue um personagem por trás precisa resolver o mesmo problema:

- Acompanhar o Player suavemente, sem ficar "grudada"
- Girar ao redor do personagem conforme o jogador movimenta o mouse
- Nunca atravessar paredes ou objetos do cenário

Sem tratamento de colisão, a câmera atravessa geometria sempre que o Player encosta em um obstáculo.

Corrigindo o Chao — MeshInstance3D × StaticBody3D

- Desde a Semana 1, Chao é apenas um MeshInstance3D — puramente visual, sem volume físico
- Nada no nível colide de verdade com ele: o Player só "parecia" parado por ter sido posicionado manualmente
- Correção: Chao passa a ser um StaticBody3D, com Malha (MeshInstance3D) e CollisionShape3D como filhos
- **StaticBody3D** — corpo físico sólido que nunca se move sozinho (cenário, paredes, chão)

SpringArm3D + Camera3D no Godot

- **CameraPivot** (`Node3D`) — filho do Player, gira no eixo Y (yaw) conforme o mouse
- **SpringArm3D** — filho do CameraPivot, gira no eixo X (pitch) e resolve a colisão da câmera automaticamente
- **Camera3D** — filho do SpringArm3D, herda a posição já resolvida
- O SpringArm3D encurta a distância sozinho quando algo bloqueia a linha até a câmera

Câmera em Terceira Pessoa no Godot × Unity

Godot

- `SpringArm3D` é um Node nativo, sem addons
- Colisão de câmera resolvida automaticamente
- Configuração simples: spring length + collision mask

Unity

- Depende do pacote **Cinemachine** (não nativo)
- Virtual Camera + Collider fazem o papel do SpringArm3D
- Mais componentes, porém mais recursos avançados (blends, damping)

Demonstração — Câmera em Terceira Pessoa

O que será construído:

- Correção do `Chao`: de `MeshInstance3D` solto para `StaticBody3D` + `CollisionShape3D`
- `CameraPivot` (`Node3D`) como filho do `Player`
- `SpringArm3D` dentro do `CameraPivot`, com Spring Length e Collision Mask configurados
- `Camera3D` dentro do `SpringArm3D`, definida como câmera ativa
- Orchestration `player.torch` capturando o mouse e rotacionando `CameraPivot` (yaw) e `SpringArm3D` (pitch, com clamp)

Por quê: sem câmera própria, o jogador não consegue de fato jogar o Vertical Slice em terceira pessoa.

Referência em GDScript — Câmera (Encontro 2)

```
extends CharacterBody3D

@onready var camera_pivot: Node3D = $CameraPivot
@onready var spring_arm: SpringArm3D = $CameraPivot/SpringArm3D

const MOUSE_SENSITIVITY := 0.003
const PITCH_MIN := deg_to_rad(-40.0)
const PITCH_MAX := deg_to_rad(60.0)

func _ready() -> void:
    Input.mouse_mode = Input.MOUSE_MODE_CAPTURED

func _unhandled_input(event: InputEvent) -> void:
    if event is InputEventMouseMotion:
        camera_pivot.rotate_y(-event.relative.x * MOUSE_SENSITIVITY)
        spring_arm.rotation.x = clamp(
            spring_arm.rotation.x - event.relative.y * MOUSE_SENSITIVITY,
            PITCH_MIN,
            PITCH_MAX
```

Imagem sugerida

*Objetivo: mostrar a árvore de cena com **Player** > **CameraPivot** > **SpringArm3D** > **Camera3D**, ao lado do Viewport 3D em modo Play, com a câmera posicionada atrás do Player.*

Enquadramento: captura de tela dividida — Scene dock à esquerda, Viewport 3D à direita.

Elementos presentes: hierarquia completa da câmera, Inspector do SpringArm3D com Spring Length e Collision Mask visíveis.

Destaque visual: seta indicando a direção do Spring Length, do CameraPivot até a Camera3D.

Legenda sugerida: "Hierarquia da câmera em terceira pessoa: CameraPivot (yaw), SpringArm3D (pitch + colisão), Camera3D."

Laboratório — Câmera em Terceira Pessoa

Cada estudante configura, no próprio projeto:

1. Converter `Chao` de `MeshInstance3D` para `StaticBody3D`, com `Malha` (`MeshInstance3D`) e `CollisionShape3D` como filhos
2. `CameraPivot` (`Node3D`) como filho do `Player`
3. `SpringArm3D` dentro do `CameraPivot`, com Spring Length e Collision Mask ajustados
4. `Camera3D` dentro do `SpringArm3D`, definida como câmera ativa
5. Captura do mouse e rotação de yaw/pitch na Orchestration `player.torch`
6. Teste com F6: girar a câmera com o mouse e encostar em uma parede do nível de teste

Boas Práticas — Câmera

- Tratar todo cenário estático (chão, paredes, plataformas) como StaticBody3D + CollisionShape3D + MeshInstance3D, nunca como MeshInstance3D solto
- Separar yaw (CameraPivot) e pitch (SpringArm3D) em Nodes distintos, nunca na mesma rotação
- Sempre aplicar clamp ao pitch, evitando que a câmera vire de cabeça para baixo
- Ajustar a Collision Mask do SpringArm3D para detectar apenas o cenário, nunca o próprio Player
- Centralizar a lógica de câmera na mesma Orchestration do Player

Semana 2 — CharacterBody3D, movimentação, Input Map e Câmera



Se o Player sempre anda no mesmo eixo do mundo, o que acontece quando eu giro a câmera 180°?

Pense no que já foi montado até aqui.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos

Tank Controls × Movimento Relativo à Câmera

- Como construído até aqui, `input_dir` é aplicado direto aos eixos globais X/Z — **tank controls** (Resident Evil clássico)
- Jogos de aventura em terceira pessoa modernos (Zelda, Dark Souls, TPS Demo do Godot) usam **movimento relativo à câmera**
- Pressionar "frente" deve mover na direção para onde a câmera está olhando, não em um eixo fixo do mundo

Corrigindo o Movimento — Basis do CameraPivot

- O vetor de direção 2D (`input_dir`) precisa ser rotacionado pela Basis do `CameraPivot` , não aplicado direto aos eixos do mundo
- `CameraPivot` só gira em Y (yaw) — sua Basis não tem pitch, então o movimento continua horizontal mesmo com a câmera inclinada
- Nunca usar a Basis do `SpringArm3D` para isso — ela contém o pitch, e inclinaria o movimento junto com a câmera

Referência em GDScript — Player Completo (Encontro 2)

```
extends CharacterBody3D

const SPEED := 5.0
const MOUSE_SENSITIVITY := 0.003
const PITCH_MIN := deg_to_rad(-40.0)
const PITCH_MAX := deg_to_rad(60.0)

@onready var camera_pivot: Node3D = $CameraPivot
@onready var spring_arm: SpringArm3D = $CameraPivot/SpringArm3D

func _ready() -> void:
    Input.mouse_mode = Input.MOUSE_MODE_CAPTURED

func _unhandled_input(event: InputEvent) -> void:
    if event is InputEventMouseMotion:
        camera_pivot.rotate_y(-event.relative.x * MOUSE_SENSITIVITY)
        spring_arm.rotation.x = clamp(
            spring_arm.rotation.x - event.relative.y * MOUSE_SENSITIVITY,
            PITCH_MIN, PITCH_MAX
```

Laboratório — Integrando Câmera e Movimento

Cada estudante ajusta, no próprio projeto:

1. Localizar os nós de `input_dir` e atribuição de `velocity` construídos no Laboratório de Input Map
2. Substituir a Basis usada no cálculo pela Basis do `CameraPivot` (não a do Player, não a do SpringArm3D)
3. Testar: girar a câmera parado, depois andar, e confirmar que o Player segue a direção da câmera
4. Testar: inclinar a câmera para cima/baixo e confirmar que o movimento continua horizontal



Desafio — Encontro 2

Adicione uma nova Action ao Input Map, não demonstrada em aula — correr, agachar ou pular.

OBJETIVOS

Liberdade de implementação: correr como multiplicador de velocidade, pular como impulso vertical simples. Não há solução única.

Resultado Esperado da Semana

- Player (CharacterBody3D) montado, com `CollisionShape3D` e `Malha` alinhados
- `Chao` convertido em `StaticBody3D`, com colisão física real
- Input Map com quatro Actions de movimentação, mais uma Action do desafio
- Câmera em terceira pessoa (`CameraPivot` + `SpringArm3D` + `Camera3D`) acompanhando o Player sem atravessar paredes
- Orchestration `player.torch` movendo o Player via `move_and_slide` **relativo à direção da câmera**, não a eixos fixos do mundo
- Turma relaciona CharacterBody3D/Input Map/SpringArm3D aos equivalentes na Unity (CharacterController, Input System, Cinemachine)

Checklist da Semana

- [] `Player` (`CharacterBody3D`) com `CollisionShape3D` e `Malha` alinhados, salvo em `scenes/characters/Player.tscn`
- [] Orchestration `player.torch` associada ao `Player`
- [] `Chao` convertido em `StaticBody3D`, com `Malha` (`MeshInstance3D`) e `CollisionShape3D` como filhos
- [] Input Map com `move_forward`, `move_back`, `move_left`, `move_right`
- [] `velocity` atribuída antes de `move_and_slide`
- [] `Player` se move nas quatro direções, sem erros
- [] `CameraPivot` + `SpringArm3D` + `Camera3D` configurados como filhos do `Player`
- [] Câmera gira com o mouse (yaw no `CameraPivot`, pitch com clamp no `SpringArm3D`) e não atravessa paredes
- [] Movimento usa a Basis do `CameraPivot` (não eixos fixos do mundo) — "frente" segue a direção da câmera

Semana 2 — CharacterBody3D, movimentação, Input Map e Câmera

Próximos Passos — Semana 3

O Player controlável desta semana é a base direta da Semana 3:

- Materiais e terreno (**Terrain3D**)
- Iluminação global (**SDFGI/VoxelGI**)
- Exportação do primeiro build executável, encerrando o Módulo 1

Leitura recomendada: Godot Docs — Physics (CharacterBody3D), Inputs, SpringArm3D e Camera3D; Unity Manual (consulta comparativa) — Input System e Cinemachine.

IFMS · Tecnologia em Jogos Digitais · Tendências de Motores de Jogos